Guia de estudo
Sistema Rastreio

Rastreabilidade e gestão da produção terceirizada de calçados

O que o sistema faz, como cada arquivo funciona e como responder às perguntas do professor.

Projeto acadêmico · SENAI Nova Serrana
Java (Spring Boot) · Vue 3 · MySQL 8 · Docker

Sumário

Como usar este guia 4

Ordem de estudo sugerida 4

Se tiver pouco tempo 4

1. O sistema em poucos minutos 5

1.1 O problema 5

1.2 A solução 5

1.3 Os quatro perfis 5

1.4 Ciclo de vida do lote 5

1.5 O exemplo oficial (decore) 5

1.6 Regras de negócio mais importantes 6

2. Tecnologias usadas 7

2.1 Backend (pasta backend/) 7

2.2 Frontend (pasta frontend/) 7

2.3 Infraestrutura e testes 8

3. Arquitetura 9

3.1 Visão geral dos containers 9

3.2 As camadas do backend 9

3.3 A jornada de uma requisição (exemplo: criar lote) 9

4. Mapa de arquivos 11

4.1 Raiz do projeto 11

4.2 Backend 11

4.3 Frontend 12

5. Backend explicado, arquivo por arquivo 14

5.1 Configuração: application.yml e pom.xml 14

5.2 Entidades (pasta entity/) 14

5.3 Repositórios (pasta repository/) 15

5.4 DTOs (dto/Entradas.java e Pagina.java) 15

5.5 Controllers (pasta controller/) 16

5.6 Services (pasta service/) 17

5.7 Segurança (pasta security/) 19

5.8 Tratamento de erros (exception/) 19

5.9 Classes de configuração e demonstração 20

6. Banco de dados 21

6.1 Diagrama entidade-relacionamento 21

6.2 Restrições que protegem os dados (V1__estrutura.sql) . . . 21
6.3 Índices . . . 22
6.4 Triggers (V2__proteger_historico.sql) . . . 22
6.5 Datas e fuso horário . . . 22

7. Frontend explicado, arquivo por arquivo . . . 23

7.1 Arquivos de base . . . 23
7.2 Componentes reutilizáveis (components/) . . . 23
7.3 Telas (views/) . . . 24
7.4 Estilo de impressão . . . 24

8. Fluxos passo a passo . . . 25

8.1 Login . . . 25
8.2 Criar o lote e gerar a etiqueta . . . 25
8.3 Ler o QR Code (câmera) . . . 25
8.4 Registrar perdas e devolver . . . 26
8.5 Cálculo do financeiro . . . 26
8.6 Painel . . . 26
8.7 Cancelar lote e cadastrar usuário . . . 26

9. Segurança, explicada . . . 27

9.1 Por que CSRF está desligado? . . . 27
9.2 Por que o token fica só em memória? . . . 27

10. Transações e concorrência . . . 28

10.1 O que é uma transação . . . 28
10.2 Prova no teste . . . 28
10.3 O problema da concorrência . . . 28
10.4 Imutabilidade do histórico em três camadas . . . 28

11. Docker . . . 29

11.1 docker-compose.yml . . . 29
11.2 Dockerfiles (multi-stage) . . . 29
11.3 Comandos úteis . . . 29

12. Testes . . . 30

12.1 Backend (pasta src/test) . . . 30
12.2 Frontend . . . 30
12.3 Como rodar . . . 30

13. O erro da primeira execução real (e a correção) . . . 31

13.1 O sintoma . . . 31
13.2 A investigação . . . 31

13.3 A causa . . . 31
13.4 A correção . . . 31
13.5 Por que os testes não pegaram? . . . 31

14. Perguntas e respostas para a apresentação . . . 33

14.1 Sobre o sistema e o negócio . . . 33
14.2 Arquitetura e código . . . 33
14.3 Segurança . . . 34
14.4 Banco de dados . . . 35
14.5 Frontend . . . 35
14.6 Docker e testes . . . 35

15. Limitações e pendências (seja honesto) . . . 37

15.1 O que foi realmente executado . . . 37
15.2 O que continua pendente (não afirmem como feito) . . . 37
15.3 Decisões a confirmar com o professor . . . 37

16. Roteiro de demonstração e cola rápida . . . 38

16.1 Antes de começar . . . 38
16.2 Roteiro sugerido (cerca de 10 a 12 minutos) . . . 38
16.3 Arquivos para abrir se o professor pedir o código . . . 38
16.4 Sugestão de divisão entre integrantes . . . 39

17. Glossário . . . 40

Como usar este guia

Este guia explica **o sistema** (o que ele faz e por quê) e **o código** (o que cada arquivo faz). Foi escrito a partir da leitura dos arquivos reais do projeto, então os nomes de classes, métodos e telas batem com o que você vai abrir na frente do professor.

Ordem de estudo sugerida

1. **Capítulo 1** – entenda o problema, os 4 perfis e o ciclo de vida do lote. Decore o exemplo 300 pares / 8 perdas / 292 aprovados / 2,67% / R$ 730,00.
2. **Capítulos 2 e 3** – tecnologias e arquitetura em camadas (Controller → Service → Repository → Banco).
3. **Capítulo 4** – mapa de todos os arquivos, para saber onde está cada coisa.
4. **Capítulos 5 a 7** – leitura do código: backend, banco de dados e frontend, arquivo por arquivo.
5. **Capítulo 8** – os fluxos passo a passo (login, criar lote, ler QR Code, perdas, financeiro).
6. **Capítulos 9 a 12** – segurança, transações/concorrência, Docker e testes.
7. **Capítulo 13** – o erro que apareceu na primeira execução real e como foi corrigido (o professor pode gostar dessa história).
8. **Capítulo 14** – perguntas e respostas prontas. Treinem em voz alta.

Se tiver pouco tempo

Leia o capítulo 1, o 3, o 8 e o 14. Com isso você consegue explicar o sistema inteiro e responder à maioria das perguntas.

> **DICA** **Os 10 pontos para saber de cabeça**
> 1) O sistema controla lotes de calçado enviados da fábrica para bancas terceirizadas, usando QR Code. 2) Quatro perfis: Administrador, Gestor, Operador da fábrica, Operador da banca. 3) O lote passa por 4 status em ordem fixa: aguardando envio → enviado à banca → em processamento → concluído (ou cancelado). 4) Backend em Java/Spring Boot, frontend em Vue 3, banco MySQL, tudo em Docker. 5) Camadas: Controller recebe, Service decide, Repository grava. 6) Login gera um token JWT; a senha é guardada com BCrypt. 7) O preço por par é copiado para o lote na criação (mudar o serviço depois não altera lotes antigos). 8) Mudança de status + histórico são gravados juntos numa transação (tudo ou nada). 9) O histórico é imutável (código sem rota de edição, entidade @Immutable e triggers no MySQL). 10) Valor devido = pares aprovados × preço por par, e só conta lote concluído.

> **ATENÇÃO** **Fale só o que vocês realmente fizeram**
> O arquivo docs/VALIDACAO.md do projeto lista o que foi e o que não foi testado. Vocês executaram o sistema no Docker com MySQL real e percorreram o fluxo manual. Se não testaram webcam, impressão da etiqueta, celular, tempos de resposta (2 s e 3 s) ou o teste E2E do Playwright, **não digam que testaram**. Digam que está implementado e que o teste físico está pendente.

1. O sistema em poucos minutos

1.1 O problema

Uma fábrica de calçados terceiriza etapas da produção (pesponto, costura, colagem) para pequenas oficinas chamadas **bancas**. Hoje isso é controlado em papel e planilha, e fica difícil saber **onde cada lote está**, **quantos pares se perderam** e **quanto pagar** a cada banca.

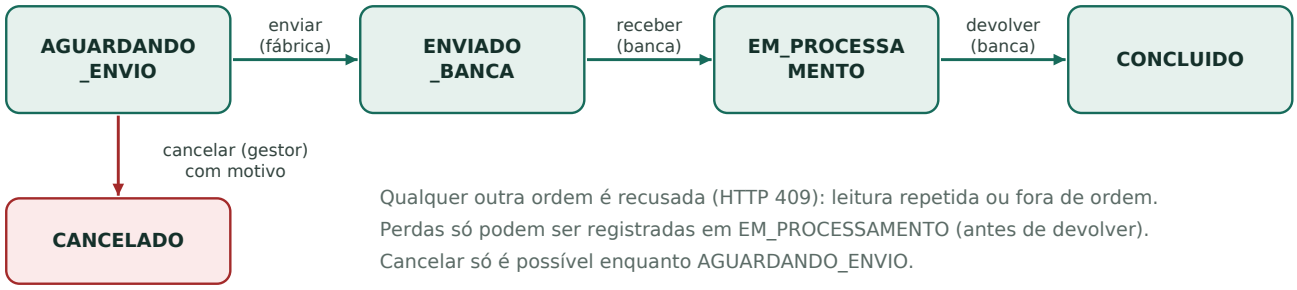
1.2 A solução

Cada lote recebe um **código único** e um **QR Code**, impresso numa etiqueta que acompanha o lote da fábrica até a banca e de volta. Em cada etapa, alguém lê o QR Code com a câmera (ou digita o código) e o sistema registra **quem fez, quando e em que ordem**. No fim, o sistema calcula automaticamente os pares aprovados, o percentual de perdas e o valor a pagar.

1.3 Os quatro perfis

Perfil	E-mail de demonstração	O que faz
Administrador	admin@demo.local	Cadastra bancas, serviços (com preço por par) e usuários. Vê painel, lotes e financeiro. Não mexe em lotes.
Gestor da fábrica	gestor@demo.local	Cria, edita e cancela lotes; imprime a etiqueta; vê painel e valores por banca.
Operador da fábrica	fabrica@demo.local	Lê o QR Code na **saída** (lote vai para a banca). Vê lotes, mas não vê valores em dinheiro.
Operador da banca	banca@demo.local	Lê o QR Code no **recebimento** e na **devolução**; registra as **perdas**. Só enxerga lotes da própria banca.

1.4 Ciclo de vida do lote



Qualquer outra ordem é recusada (HTTP 409): leitura repetida ou fora de ordem.
Perdas só podem ser registradas em EM_PROCESSAMENTO (antes de devolver).
Cancelar só é possível enquanto AGUARDANDO_ENVIO.

Esse ciclo está codificado em um único lugar: o método `Calculos.transicao`. Ele recebe o status atual e a ação, confere se aquela ação é permitida naquele status e devolve o novo status. Qualquer combinação fora da tabela dispara um erro 409 ("Leitura repetida ou fora de ordem").

1.5 O exemplo oficial (decore)

Passo	Quem	O que acontece	Estado do lote
1	Gestor	Cria lote de 300 pares (grade 38, 39, 40), serviço Pesponto a R$ 2,50	AGUARDANDO_ENVIO
2	Op. fábrica	Lê o QR Code (saída)	ENVIADO_BANCA
3	Op. banca	Lê o QR Code (recebimento)	EM_PROCESSAMENTO

Passo	Quem	O que acontece	Estado do lote
4	Op. banca	Registra 8 pares danificados	continua EM_PROCESSAMENTO, perdas = 8
5	Op. banca	Lê o QR Code (devolução)	CONCLUIDO
6	Sistema	Aprovados = 300 − 8 = **292**; perdas = 8 ÷ 300 = **2,67%**; valor = 292 × 2,50 = **R$ 730,00**	histórico com 4 eventos

1.6 Regras de negócio mais importantes

- O lote só pode ser **editado ou cancelado antes de sair da fábrica** (status aguardando envio). O cancelamento exige motivo.
- O **serviço de um lote não pode mudar** depois de criado (cancele e crie outro).
- O **preço por par é copiado** do serviço para o lote na criação. Mudar o preço do serviço depois só vale para lotes novos.
- A soma das **perdas não pode passar do total de pares** do lote. Perdas só no status EM_PROCESSAMENTO.
- Cada etapa só pode ocorrer **uma vez e na ordem certa**. Repetir uma leitura é recusado e não cria evento novo.
- O operador da banca **só acessa lotes da própria banca**. Lote de outra banca responde "não encontrado".
- O **histórico não pode ser editado nem apagado**.
- **Valor devido** = aprovados × preço contratado; só **lotes concluídos** viram dívida. Lote aberto mostra apenas uma previsão.
- Tudo em português do Brasil, datas dd/mm/aaaa e **horário de Brasília**.

2. Tecnologias usadas

O requisito do trabalho é: **Java (Spring Boot) + Vue.js + MySQL 8 + Docker**. Abaixo, cada peça e para que ela serve aqui.

2.1 Backend (pasta backend/)

Tecnologia	Para que serve no projeto
Java 17	Linguagem do servidor. (O `pom.xml` usa `java.version` 17.)
Maven (pom.xml)	Baixa as bibliotecas e compila o projeto em um arquivo `.jar`.
Spring Boot 3.5.7	Framework que sobe o servidor web, liga as classes entre si (injeção de dependência) e configura tudo por convenção.
Spring Web (REST)	Permite criar as rotas `/api/...` com `@RestController`, `@GetMapping`, `@PostMapping`.
Spring Data JPA + Hibernate	Transformam classes Java (`@Entity`) em tabelas e escrevem o SQL básico automaticamente.
Bean Validation (`@Valid`)	Valida os dados que chegam (`@NotBlank`, `@Min`, `@FutureOrPresent`...).
Spring Security + OAuth2 Resource Server	Protege a API e valida o token JWT em cada requisição.
BCrypt	Embaralha a senha de forma irreversível antes de guardar no banco.
Flyway	Cria e versiona as tabelas com os scripts `V1__estrutura.sql` e `V2__proteger_historico.sql`.
MySQL 8.0 + driver	Banco de dados relacional onde tudo é gravado.
ZXing	Biblioteca que gera a imagem PNG do QR Code.
springdoc / Swagger	Gera a documentação interativa em `/swagger-ui.html`.
Spring Actuator	Expõe `/actuator/health`, usado para saber se a API está viva.
JdbcTemplate	SQL manual (agregações do painel e do financeiro).

2.2 Frontend (pasta frontend/)

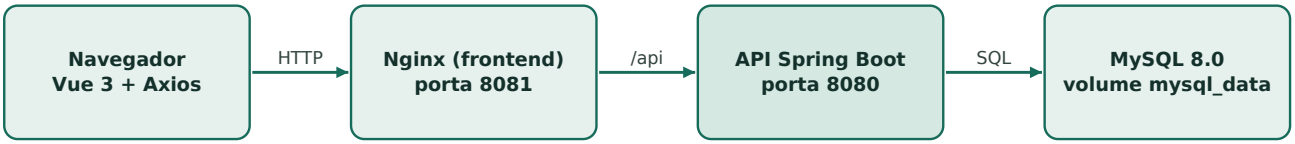
Tecnologia	Para que serve no projeto
Vue 3 (`<script setup>`)	Monta as telas com componentes reativos (a tela se atualiza quando os dados mudam).
Vite	Servidor de desenvolvimento e empacotador (`npm run dev`, `npm run build`).
Vue Router	Navegação entre telas (`/lotes`, `/leitor`...) e bloqueio por perfil.
Pinia	Guarda o estado global: aqui, o token e o usuário logado (`stores/auth.js`).
Axios	Faz as chamadas HTTP para `/api` (`api.js`).
Chart.js	Desenha os gráficos do painel.
html5-qrcode	Lê o QR Code pela câmera, direto no navegador.
Vitest + Vue Test Utils	Testes automatizados do frontend.
Playwright	Teste E2E (navegador de verdade). O arquivo existe em `e2e/fluxo.spec.js`.

2.3 Infraestrutura e testes

Tecnologia	Para que serve no projeto
Docker + Docker Compose	Sobe MySQL, backend e frontend com um único `docker compose up`.
Nginx	Serve o frontend compilado e repassa `/api` para o backend.
JUnit 5, Spring Boot Test, MockMvc	Testes do backend (simulam chamadas HTTP sem abrir navegador).
Mockito	Simula falhas (ex.: falha ao gravar histórico) para provar o rollback.
H2	Banco de memória usado só nos testes automáticos.
Testcontainers	Sobe um MySQL real dentro dos testes (`MySqlTest`), quando habilitado.
Git	Controle de versão (o projeto inclui `historico-git.bundle`).

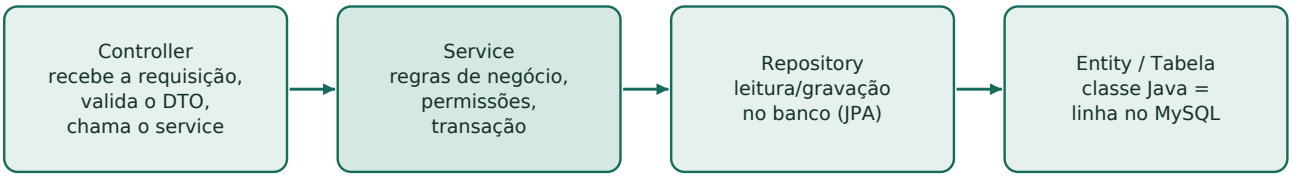
3. Arquitetura

3.1 Visão geral dos containers



Tudo roda em 3 containers Docker (mysql, backend, frontend), criados pelo docker-compose.yml.
O navegador só fala com o Nginx (mesma origem). O Nginx entrega a tela e repassa /api para o Java.
Só o Java conversa com o banco. A senha do banco e o segredo do token ficam no arquivo .env.

3.2 As camadas do backend



Ao lado: Acesso/Seguranca (quem é você e o que pode), Erros (respostas seguras) e Calculos (contas puras).
A resposta volta pelo mesmo caminho: Entity → Service (monta o resumo) → Controller (JSON).

Camada	Pasta / arquivos	Responsabilidade	Analogia
Controller	controller/*.java	Define a rota (`/api/lotes`), converte JSON em objeto (DTO), valida e devolve o código HTTP.	Recepção do prédio
DTO	dto/Entradas.java, Pagina.java	Formato dos dados que entram e saem. Impede que o cliente mande status ou preço.	Formulário de entrada
Service	service/*.java	Regras de negócio, permissões, cálculos, transações.	Gerente que decide
Repository	repository/*.java	Ler e gravar no banco, sem escrever SQL na mão.	Arquivista
Entity	entity/*.java	Cada classe é uma tabela; cada objeto, uma linha.	Ficha de papel
Segurança	security/*.java	Quem é o usuário (JWT/BCrypt) e o que ele pode fazer.	Porteiro e crachá
Erros	exception/*.java	Converte falhas em respostas seguras, em português.	Tradutor de problemas
Config	config/*.java	Swagger, fuso horário da validação, dados de demonstração.	Ajustes

3.3 A jornada de uma requisição (exemplo: criar lote)

1. O gestor preenche o formulário em `LoteFormView.vue` e clica em **Criar lote e gerar QR Code**.
2. A função `salvar()` monta o JSON (com a grade `{"38":100,"39":100,"40":100}`) e chama `api.post('/lotes', body)`.
3. O interceptor do Axios (em `main.js`) adiciona o cabeçalho `Authorization: Bearer <token>`.
4. O Nginx recebe `/api/lotes` e repassa para `backend:8080`.

5. O Spring Security valida o token (assinatura, emissor `rastreio` e validade). Se falhar, responde 401.
6. `LoteController.criar` recebe o JSON como `LoteInput`; o `@Valid` checa os campos (400 se algo estiver errado).
7. `LoteService.salvar` confere que o usuário é GESTOR, que banca e serviço estão ativos, soma a grade, gera o código `L-ANO-UUID`, copia o preço do serviço e salva.
8. Na mesma transação, grava o evento CRIACAO no histórico.
9. O service devolve um resumo do lote; o controller responde 201 com JSON.
10. A tela recebe o código e navega para `/lotes/<código>`, onde baixa o QR Code e mostra o detalhe.

4. Mapa de arquivos

Estrutura da pasta `projeto-calcados/`. Os caminhos Java abaixo partem de `backend/src/main/java/br/edu/senai/rastreio/`.

4.1 Raiz do projeto

Arquivo / pasta	Para que serve
`docker-compose.yml`	Define os 3 serviços (mysql, backend, frontend), portas, volume e variáveis.
`.env.example` → `.env`	Modelo das configurações secretas (senhas, JWT). O `.env` real nunca vai para o Git.
`.gitignore`	Lista o que o Git ignora (ex.: `.env`, `node_modules`, `target`).
`README.md`	Como subir o sistema do zero, usuários demo, roteiro de teste.
`historico-git.bundle`	Cópia do repositório Git com as branches main, develop, feature/sistema e test/testes-integracao.
`infra/Caddyfile.example`	Exemplo de proxy HTTPS (a câmera exige HTTPS fora do localhost).
`docs/`	Documentação: requisitos, casos de uso, diagramas, API, testes manuais, validação, Scrum.
`backend/`	API em Java (Spring Boot).
`frontend/`	Interface em Vue 3.

4.2 Backend

Arquivo	Papel	Importância
`pom.xml`	Dependências e versão do Java/Spring.	Média
`Dockerfile`	Compila com Maven e roda o `.jar` com usuário sem privilégios.	Média
`RastreioApplication.java`	Ponto de entrada (`main`). `@SpringBootApplication`.	Baixa
`controller/AuthController.java`	Login (`POST /api/auth/login`) e `/api/auth/me`.	Alta
`controller/LoteController.java`	Rotas de lotes, leitura, perdas, cancelamento e QR Code.	**Muito alta**
`controller/CadastroController.java`	Listagens e cadastros de bancas, serviços e usuários.	Alta
`controller/PainelController.java`	`/api/dashboard` e `/api/relatorios/financeiro`.	Média
`service/LoteService.java`	**Coração do sistema**: criar/editar, movimentar, perdas, histórico.	**Muito alta**
`service/Calculos.java`	Aprovados, percentual, valor e tabela de transições (regras puras).	**Muito alta**
`service/CadastroService.java`	Regras de cadastro de bancas, serviços e usuários.	Alta
`service/PainelService.java`	SQL agregado do painel e do financeiro.	Alta
`service/Documentos.java`	Valida CPF/CNPJ pelos dígitos verificadores.	Baixa
`security/Seguranca.java`	BCrypt, JWT e quais rotas são públicas.	**Muito alta**
`security/Acesso.java`	Descobre o usuário atual e confere perfil e banca.	**Muito alta**

Arquivo	Papel	Importância
`entity/*.java` (9 arquivos)	Banca, Servico, Usuario, Lote, Movimentacao, Perda + enums Perfil, Status, TipoPerda.	Alta
`repository/*.java` (5)	Interfaces de acesso ao banco; `LoteRepository` tem o bloqueio.	Média
`dto/Entradas.java`	Formatos de entrada validados (records).	Alta
`dto/Pagina.java`	Formato padrão de página (content, totalPages...).	Baixa
`exception/Erros.java`	Trata todas as falhas e devolve mensagens seguras.	Alta
`exception/RegraException.java`	Erro de negócio com código HTTP.	Média
`config/Demonstracao.java`	Cria os 4 usuários, 1 banca e 1 serviço de demonstração na primeira subida.	Média
`config/ValidacaoConfig.java`	Valida datas no fuso de Brasília.	Baixa
`config/OpenApiConfig.java`	Configura o Swagger com autenticação Bearer.	Baixa
`resources/application.yml`	Configurações (banco, JWT, fuso, Flyway, Swagger).	Alta
`resources/db/migration/V1__estrutura.sql`	Cria todas as tabelas, restrições e índices.	Alta
`resources/db/migration/V2__proteger_historico.sql`	Triggers que proíbem alterar/apagar histórico e perdas.	Alta
`src/test/...`	Testes: `CalculosTest`, `FluxoBase`, `IntegracaoTest`, `AtomicidadeTest`, `ConcorrenciaTest`, `MySqlTest`.	Alta

4.3 Frontend

Arquivo	Papel	Importância
`index.html`	Página única; carrega `src/main.js`.	Baixa
`package.json`	Dependências e scripts (`dev`, `build`, `test`, `test:e2e`).	Média
`vite.config.js`	Plugin Vue, proxy `/api` → `localhost:8080` e configuração de testes.	Média
`nginx.conf`	Serve o site, trata rotas internas (SPA) e repassa `/api` ao backend.	Média
`Dockerfile`	Compila com Node e serve com Nginx.	Média
`src/main.js`	Inicia o Vue, Pinia, rotas e os interceptores do Axios (token e 401).	**Muito alta**
`src/App.vue`	Estrutura visual (menu lateral por perfil, topo, rodapé).	Alta
`src/router.js`	Rotas e bloqueio por perfil (`meta.roles`).	**Muito alta**
`src/api.js`	Instância do Axios, `parametros()` e `catalogo()`.	Alta
`src/stores/auth.js`	Estado de login (token e usuário) em memória.	Alta
`src/composables/useTask.js`	Controle de carregando/erro/sucesso e bloqueio de duplo clique.	Alta
`src/scanGate.js`	Trava que impede ler o mesmo QR várias vezes seguidas.	Alta
`src/utils.js`	Formatação de moeda/data (Brasília), nomes de status e perfis.	Média

Arquivo	Papel	Importância
`src/style.css`	Todo o visual (inclui estilo de impressão da etiqueta de 40 mm).	Baixa
`views/LoginView.vue`	Tela de login.	Média
`views/DashboardView.vue`	Painel com indicadores, gráficos e atualização a cada 15 s.	Alta
`views/LotesView.vue`	Lista de lotes com filtros e paginação.	Média
`views/LoteFormView.vue`	Criar/editar lote (dados e grade).	Alta
`views/LoteDetailView.vue`	Detalhe, etiqueta/QR, perdas, linha do tempo, cancelamento.	**Muito alta**
`views/LeitorView.vue`	Câmera e código manual para saída, recebimento e devolução.	**Muito alta**
`views/FinanceiroView.vue`	Valores por banca e mês.	Média
`views/CadastroView.vue`	Cadastro de bancas, serviços e usuários (uma tela para os três).	Média
`components/*.vue`	Filtros, Gráfico, Paginação, Feedback (mensagens) e StatusBadge.	Média
`*.test.js`, `e2e/fluxo.spec.js`	Testes do frontend e teste E2E.	Média

5. Backend explicado, arquivo por arquivo

Todos os caminhos partem de `backend/src/main/java/br/edu/senai/rastreio/`. Os arquivos mais cobrados estão marcados como **Muito alta**, no capítulo 4.

5.1 Configuração: application.yml e pom.xml

O `application.yml` diz ao Spring como se conectar e se comportar. Os valores com `${...}` vêm de **variáveis de ambiente** (no Docker, do arquivo `.env`), então nenhuma senha fica dentro do código.

Configuração	O que significa
`spring.datasource.url/username/password`	Endereço e credenciais do MySQL. `DB_PASSWORD` é obrigatória.
`jpa.hibernate.ddl-auto: validate`	O Hibernate **não cria** tabelas; só confere se o banco bate com as entidades. Quem cria é o Flyway.
`jpa.open-in-view: false`	Evita deixar a conexão aberta durante a renderização da resposta (boa prática).
`hibernate.jdbc.time_zone: UTC`	Datas são gravadas em UTC; a conversão para Brasília é feita na exibição.
`jackson.time-zone: America/Sao_Paulo`	As datas do JSON saem no fuso de Brasília.
`server.error.include-message: never`	Nunca vaza mensagem interna de erro para o navegador.
`app.jwt-secret`, `app.token-minutes`	Segredo que assina o token e tempo de validade (padrão 60 minutos).
`app.demo-enabled`, `app.demo-password`	Ligam a carga dos dados de demonstração e definem a senha deles.
`management.endpoints...include: health`	Só o endpoint `health` do Actuator fica exposto.

5.2 Entidades (pasta entity/)

Uma **entidade** é uma classe Java ligada a uma tabela. Os campos são públicos (estilo simples escolhido pelo projeto). Anotações importantes:

Anotação	Significado
`@Entity`, `@Table(name=...)`	Esta classe é uma tabela com esse nome.
`@Id` + `@GeneratedValue(IDENTITY)`	Chave primária com auto incremento do MySQL.
`@Column(nullable, unique, length)`	Regras da coluna (obrigatória, única, tamanho).
`@ManyToOne` + `@JoinColumn`	Vínculo "muitos para um" (vários lotes → uma banca). Cria a chave estrangeira.
`@ElementCollection` (Lote.grade)	Guarda um mapa numeração → quantidade na tabela `lote_grade`.
`@Enumerated(STRING)`	Grava o nome do enum (ex.: `CONCLUIDO`) em vez de um número.
`@Version` (Lote.versao)	Controle de versão: se dois usuários alterarem o mesmo lote, o segundo recebe conflito.
`@Immutable` (Movimentacao, Perda)	O Hibernate nunca faz UPDATE nessas linhas. `updatable = false` nas colunas reforça.

Entidade	Tabela	Campos principais	Observações
Banca	bancas	nome, documento (único), responsavel, telefone, ativo	Desativação lógica (`ativo`), nunca exclusão.
Servico	servicos	nome, descricao, valorPar (BigDecimal), ativo	O preço atual; o lote guarda uma **cópia**.
Usuario	usuarios	nome, email (único), senha, perfil, banca, ativo	`senha` tem `@JsonIgnore`: o hash **nunca** vai no JSON.
Lote	lotes	codigo (único), programacao, modelo, cor, banca, servico, prazo, grade, totalPares, totalPerdas, valorPar, status, datas, versao	O código é imutável; `valorPar` é o preço contratado.
Movimentacao	movimentacoes	lote, usuario, responsavel, anterior, posterior, tipo, ocorridoEm, observacao	Histórico imutável. Guarda também o **nome** do responsável em texto.
Perda	perdas	lote, usuario, responsavel, tipo, quantidade, observacao, ocorridoEm	Também imutável.
Perfil (enum)	—	ADMIN, GESTOR, OPERADOR_FABRICA, OPERADOR_BANCA	
Status (enum)	—	AGUARDANDO_ENVIO, ENVIADO_BANCA, EM_PROCESSAMENTO, CONCLUIDO, CANCELADO	
TipoPerda (enum)	—	DANIFICADA, EXTRAVIADA, DEFEITO_FABRICACAO	

5.3 Repositórios (pasta repository/)

Interfaces que **não têm código dentro**: ao estender `JpaRepository`, o Spring Data gera `save`, `findById`, `findAll`, `count` e outras. Também estendem `JpaSpecificationExecutor`, que permite filtros dinâmicos montados em código (usado nas listagens).

LoteRepository.java (o trecho mais importante dos repositórios)

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select l from Lote l where l.codigo = :codigo")
Optional<Lote> bloquear(@Param("codigo") String codigo);
Optional<Lote> findByCodigo(String codigo);
```

`bloquear` executa um `SELECT ... FOR UPDATE`: enquanto uma transação estiver mexendo naquele lote, outra que tente bloqueá-lo **espera**. É isso que impede duas perdas simultâneas de estourarem o limite (capítulo 10). `findByCodigo` é a busca comum, sem bloqueio. `UsuarioRepository.findByEmail` serve ao login e `MovimentacaoRepository`/`PerdaRepository` listam o histórico de um lote em ordem.

5.4 DTOs (dto/Entradas.java e Pagina.java)

DTO (Data Transfer Object) é o formato dos dados que entram na API. São `record`s com validações. O ponto de segurança: o DTO de lote **não tem** campos de status, preço nem totais, então o cliente não consegue enviá-los (mesmo que tente, são ignorados).

DTO	Campos e validações principais
Login	`email` (`@Email`), `senha` (máx. 72, limite do BCrypt).
BancaInput	nome, documento, responsável, telefone (`@NotBlank` + tamanhos), ativo.
ServicoInput	nome, descrição, `valorPar` (`@DecimalMin(0.01)`, 2 casas), ativo.
UsuarioInput	nome, email, senha (opcional na edição), perfil, bancaId, ativo.

DTO	Campos e validações principais
LoteInput	programação, modelo, cor, bancaId, servicoId, `prazo` (`@FutureOrPresent`), `grade` (mapa numeração → quantidade, 1 a 100 itens, cada quantidade 0 a 1.000.000).
Cancelamento	`motivo` obrigatório (até 500 caracteres).
PerdaInput	`tipo`, `quantidade` (1 a 100.000.000), `observacao` obrigatória.
Leitura	`codigo` (até 50 caracteres).
Pagina<T>	Formato estável de paginação: `content`, `number`, `size`, `totalElements`, `totalPages`.

5.5 Controllers (pasta controller/)

Os controllers são finos: recebem, validam e **delegam ao service**. As permissões e regras ficam no service (assim valem para qualquer entrada).

LoteController: /api/lotes

Método e rota	Função no controller	Chama
GET /api/lotes	`listar` (filtros: busca, status, bancaId, inicio, fim, page, size)	`LoteService.listar`
GET /api/lotes/{codigo}	`detalhe`	`LoteService.detalhe`
POST /api/lotes	`criar` (retorna 201)	`LoteService.salvar(null, dto)`
PUT /api/lotes/{codigo}	`editar`	`LoteService.salvar(codigo, dto)`
POST /api/lotes/{codigo}/cancelar	`cancelar`	`movimentar(codigo, "cancelar", motivo)`
POST /api/lotes/leitura/{acao}	`ler`: aceita só `enviar`, `receber`, `devolver` (senão 400)	`movimentar(codigo, acao, null)`
POST /api/lotes/{codigo}/perdas	`perda` (retorna 201)	`LoteService.perda`
GET /api/lotes/{codigo}/qrcode	`qr`: gera o PNG	`detalhe` (para checar acesso) + ZXing

Como o QR Code é gerado (método `qr`): primeiro chama `s.detalhe(codigo)`, que lança erro se o usuário não puder ver aquele lote (assim ninguém gera QR de lote alheio). Depois usa `MultiFormatWriter` do ZXing com formato `QR_CODE`, 480×480 pixels, **correção de erro nível H** (lê mesmo amassado) e margem 4, e `MatrixToImageWriter` escreve o PNG. O conteúdo do QR é **somente o código do lote**: sem preço, sem CPF/CNPJ, sem token.

AuthController: /api/auth

AuthController.login (resumido)

```
var u = usuarios.findByEmail(entrada.email().trim().toLowerCase())
      .orElseThrow(() -> new RegraException(401, "E-mail ou senha inválidos."));
if (!u.ativo || !senhas.matches(entrada.senha(), u.senha))
  throw new RegraException(401, "E-mail ou senha inválidos.");
var claims = JwtClaimsSet.builder().issuer("rastreio")
      .subject(u.id.toString()).issuedAt(agora)
      .expiresAt(agora.plusSeconds(minutos * 60)).build();
```

Pontos para explicar: (1) e-mail inexistente, senha errada e usuário desativado dão **a mesma mensagem**, para não revelar quais e-mails existem; (2) `senhas.matches` compara a senha digitada com o hash BCrypt; (3) o token JWT contém só **quem é** (`subject` = id do usuário), **quem emitiu** e **quando expira**, e é assinado com HS256; (4) a resposta devolve `token`, `usuario` (sem senha) e `expiraEm`; (5) `GET /me` devolve o usuário atual.

CadastroController: /api/bancas, /api/servicos, /api/usuarios

GET lista com paginação e busca por nome (`LIKE` em minúsculas). Regras de acesso: **bancas** qualquer logado (o operador de banca só vê a sua); **serviços** só Admin e Gestor; **usuários** só Admin. POST cria (201) e PUT edita, delegando ao `CadastroService`. Não existe DELETE: cadastros são desativados.

> **ATENÇÃO Este arquivo foi corrigido na apresentação**
> A versão original lia `s.bancas`, `s.servicos` e `s.usuarios` (repositórios dentro do `CadastroService`) e causava erro 500. A versão corrigida injeta os três repositórios direto no construtor do controller. A história completa está no capítulo 13.

PainelController

`GET /api/dashboard` (filtros inicio, fim, bancaId, status) e `GET /api/relatorios/financeiro` (inicio, fim, bancaId, page, size). Ambos só delegam ao `PainelService`.

5.6 Services (pasta service/)

LoteService: o coração do sistema

Tem a anotação `@Service` e usa 6 dependências (repositórios de lote, banca, serviço, movimentação, perda e o `Acesso`). A constante `BRASIL` (`America/Sao_Paulo`) é usada em todas as contas de data. Método por método:

Método	O que faz
`filtro(...)`	Monta a consulta dinâmica da lista. Se o usuário é OPERADOR_BANCA, **sempre** acrescenta `banca = banca do usuário`. Aplica busca por código (maiúsculas), status, banca e período (início inclusivo, fim exclusivo no dia seguinte, em Brasília). Valida que fim não é antes do início.
`listar(...)`	Executa o filtro com paginação (tamanho máx. 100, ordem do mais novo para o mais antigo) e converte cada lote em `resumo`.
`consultar(codigo, bloquear)`	Busca o lote (com ou sem bloqueio) e chama `acesso.lote(...)`. Lote inexistente **ou** de outra banca vira 404 com a mesma mensagem.
`detalhe(codigo)`	`@Transactional(readOnly = true)`. Devolve o resumo + `historico` (linha do tempo) + `perdas`.
`resumo(lote)`	Monta o mapa que vai para o JSON: dados, grade, totais, **aprovados**, **percentualPerdas**, status e datas. Se o usuário **não** for operador da fábrica, inclui `valorPar`, `valorPrevisto` e `valorDevido` (este só se concluído).
`salvar(codigo, dto)`	Cria (codigo nulo) ou edita um lote. Detalhado abaixo.
`movimentar(codigo, acao, motivo)`	Enviar, receber, devolver ou cancelar. Detalhado abaixo.
`perda(codigo, dto)`	Registra perda. Detalhado abaixo.
`historico(...)` (privado)	Cria o evento (lote, usuário, nome do responsável, status anterior e posterior, tipo, observação) e grava com `saveAndFlush`.

salvar: criar e editar lote

1. `acesso.exigir(Perfil.GESTOR)`: só gestor.
2. Se for edição, carrega o lote **com bloqueio**; se o status não for AGUARDANDO_ENVIO, erro 409.
3. Busca banca e serviço e exige que estejam **ativos** (senão 400).
4. Na edição, se tentar trocar o serviço, erro 409 (o preço contratado ficaria confuso).
5. Soma a grade: `d.grade().values().stream().mapToInt(Integer::intValue).sum()`; total deve estar entre 1 e 100 milhões.

6. Prazo não pode ser anterior a hoje em Brasília.
7. Copia os campos (com `trim()`), substitui a grade e define `totalPares`.
8. Na **criação**: gera o código `"L-" + ano + "-" + UUID sem traços em maiúsculas` e **copia** `s.valorPar` **para** `l.valorPar`.
9. `lotes.save(l)` e depois `historico(...)` com tipo CRIACAO ou EDICAO.

Por que UUID no código? Um contador sequencial (`L-2026-000123`) exigiria controlar concorrência entre dois gestores criando lotes ao mesmo tempo. O UUID é único sem coordenação, e a restrição `UNIQUE` do banco é a última defesa.

movimentar: enviar, receber, devolver, cancelar

LoteService.movimentar (resumido)

```
var u = switch (acao) {
  case "enviar" -> acesso.exigir(Perfil.OPERADOR_FABRICA);
  case "receber", "devolver" -> acesso.exigir(Perfil.OPERADOR_BANCA);
  case "cancelar" -> acesso.exigir(Perfil.GESTOR);
  default -> throw new RegraException(400, "Ação inválida.");
};
var l = consultar(codigo, true);          // bloqueia o lote
var anterior = l.status;
var novo = Calculos.transicao(anterior, acao); // valida a ordem
l.status = novo;
if (acao.equals("enviar"))   l.enviadoEm = Instant.now();
if (acao.equals("devolver")) l.concluidoEm = Instant.now();
historico(l, u, anterior, acao.toUpperCase(), motivo);
```

Repare: **não há** `lotes.save(l)`. Como o método é `@Transactional` e `l` é uma entidade gerenciada, o JPA detecta que o status mudou e grava no fim da transação (*dirty checking*). Se o `historico` falhar, a transação inteira é desfeita e o status volta ao que era (é o que o `AtomicidadeTest` prova).

perda: registrar perdas

1. Só OPERADOR_BANCA.
2. Carrega o lote **com bloqueio** (e confere que é da banca do usuário).
3. Status precisa ser EM_PROCESSAMENTO; senão 409 ("Perdas só podem ser registradas durante o processamento").
4. `Calculos.aprovados(total, perdasAtuais + quantidade)`: se passar do total, lança 409.
5. Cria a `Perda` (com o nome do responsável), salva e soma em `l.totalPerdas`.

Calculos: regras puras

service/Calculos.java

```
aprovados(total, perdas)  = total - perdas          (409 se perdas > total ou perdas < 0)
percentual(total, perdas) = perdas * 100 / total    (2 casas, HALF_UP)
valor(total, perdas, preco) = preco * aprovados     (2 casas, HALF_UP)
transicao(atual, acao)    = ver tabela do ciclo de vida
```

É uma classe sem Spring e sem banco, por isso é fácil de testar (`CalculosTest`). Usa `BigDecimal` para dinheiro: `double` erra centavos (0,1 + 0,2 não é 0,3). `HALF_UP` é o arredondamento escolar (2,666... vira 2,67).

CadastroService

- **banca**: só Admin; remove máscara do CPF/CNPJ e valida com `Documentos.valido`; telefone precisa ter 10 a 13 dígitos; cria ou edita.
- **servico**: só Admin; nome, descrição, valor por par, ativo.
- **usuario**: só Admin; não deixa o admin remover o **próprio** acesso (desativar ou mudar o próprio perfil); senha inicial obrigatória na criação; senha de **10 a 72 bytes**, gravada com BCrypt; operador de banca **exige banca ativa**; e-mail em minúsculas.

- Todos usam `@Transactional` e **desativação lógica** (campo `ativo`) para preservar o histórico.

PainelService

Usa `NamedParameterJdbcTemplate` (SQL escrito à mão, com parâmetros nomeados como `:inicio`) porque agregações (`COUNT`, `SUM`, `AVG`) são mais eficientes no banco do que carregar todos os lotes na memória. A classe é `@Transactional(readOnly = true)`.

Método / consulta	Resultado
`filtro(...)`	Monta o `WHERE` com período, banca e status. O nome da coluna de data (`criado_em` ou `concluido_em`) vem de constantes internas; os **valores** do usuário sempre vão por parâmetros (sem SQL injection).
`painel` → indicadores	Lotes, pares, perdas, % de perdas, valor devido (só concluídos), ciclo médio em horas (`TIMESTAMPDIFF` entre envio e conclusão) e lotes atrasados (prazo < hoje e não concluído/cancelado).
`painel` → listas	Lotes por status, por banca, perdas por tipo, evolução dos últimos 30 dias, valores por banca e os 20 lotes mais atrasados.
`financeiro`	Só CONCLUÍDOS, agrupados por **banca + ano + mês da conclusão** (convertendo UTC para UTC−3), com paginação, total de grupos e `totalDevido`.

Documentos

Valida CPF (11 dígitos) e CNPJ (14) calculando os **dois dígitos verificadores** com pesos, e recusa sequências repetidas como `11111111111`. Teste: `11222333000181` é válido e `11222333000182` não.

5.7 Segurança (pasta security/)

Seguranca.java

Bean / trecho	Função
`PasswordEncoder = BCryptPasswordEncoder`	Cria e compara hashes de senha.
`SecretKeySpec chave`	Lê `JWT_SECRET`; **recusa** menos de 32 bytes (a aplicação nem inicia).
`JwtEncoder` / `JwtDecoder`	Assinam e validam tokens HS256. O decoder exige `issuer = rastreio` e checa a expiração.
`SecurityFilterChain`	CSRF desligado (sem cookies de sessão); sessão **STATELESS**; rotas públicas: login, `/actuator/health`, `/error`, Swagger; **todo o resto exige autenticação**; respostas 401 em JSON em português.

Acesso.java

security/Acesso.java

```
atual()  -> lê o id do token, busca o usuário NO BANCO e exige ativo (senão 401)
exigir() -> confere se o perfil está na lista permitida (senão 403)
lote()   -> OPERADOR_BANCA só passa se lote.banca == usuario.banca (senão 404)
```

Detalhe importante: o perfil **não vem do token**; é reconsultado no banco a cada operação. Por isso, desativar um usuário ou mudar seu perfil vale **imediatamente**, mesmo com token ainda válido.

5.8 Tratamento de erros (exception/)

Situação	Código	Mensagem ao usuário
`RegraException` (regra de negócio)	o definido na regra	A mensagem da própria regra

Situação	Código	Mensagem ao usuário
Campos inválidos (`@Valid`)	400	"Revise os campos informados." + lista de campos
JSON ilegível / filtro inválido	400	"Dados ou filtros inválidos."
Duplicidade ou conflito de concorrência	409	"Registro duplicado, vinculado ou alterado por outro usuário..."
Acesso negado	403	"Acesso não autorizado."
Qualquer falha inesperada	500	"Não foi possível concluir. Tente novamente." (o detalhe vai só para o log)

`Erros.java` é um `@RestControllerAdvice`: captura exceções de **todos** os controllers. O ponto de segurança é nunca devolver SQL, senha ou *stack trace* ao navegador. A causa real é registrada no log do servidor (a linha `Falha interna`, que usamos para achar o bug).

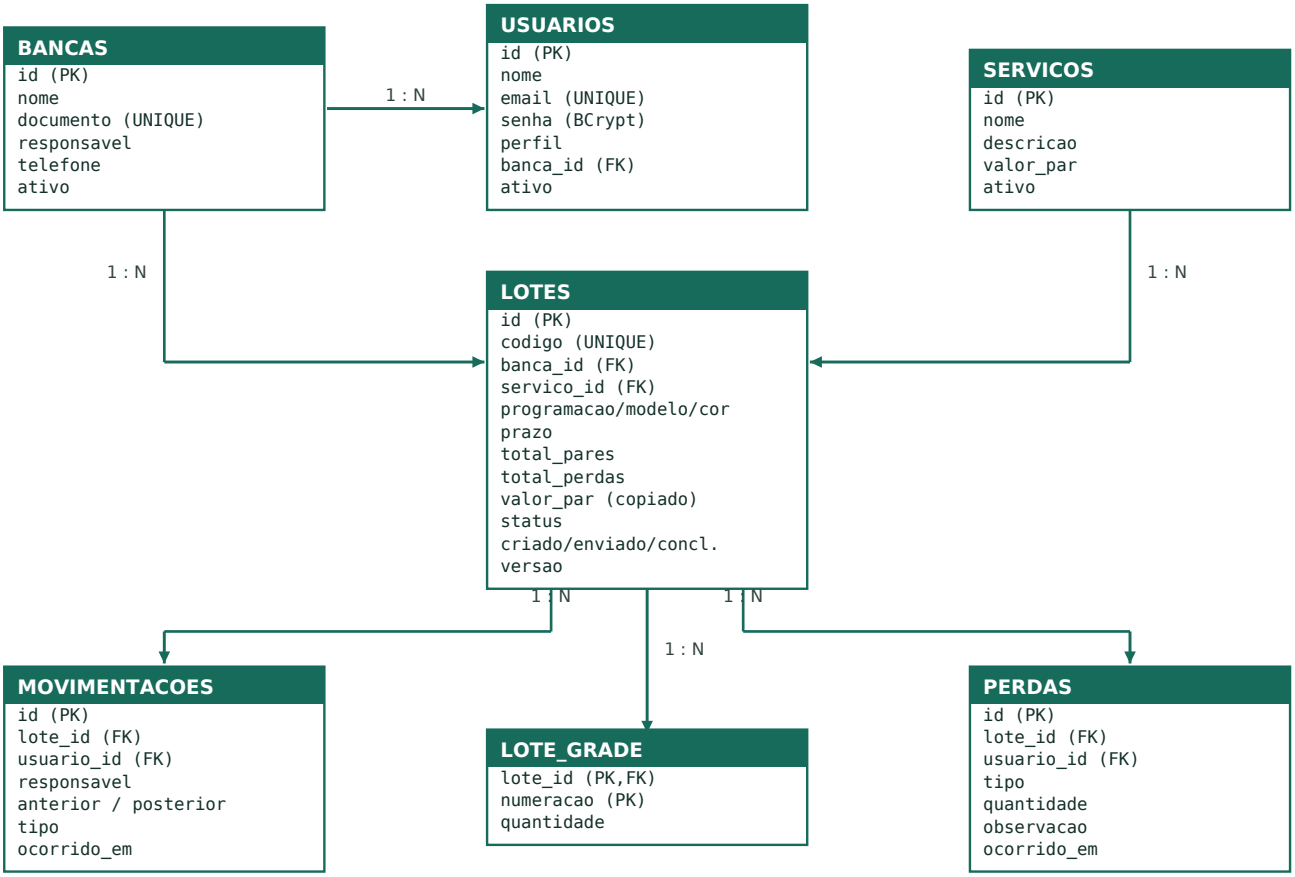
5.9 Classes de configuração e demonstração

- **Demonstracao** (`CommandLineRunner`): roda uma vez ao subir. Se `DEMO_ENABLED=false`, não faz nada. Exige senha de 10+ caracteres. **Só cria dados se a tabela de usuários estiver vazia** (idempotente: reiniciar não duplica nem sobrescreve). Cria 1 banca, 1 serviço (Pesponto, R$ 2,50) e 1 usuário por perfil, com a senha de `DEMO_PASSWORD` em BCrypt; o operador de banca é ligado à banca de demonstração.
- **ValidacaoConfig**: faz o Bean Validation usar o relógio de Brasília, para o `@FutureOrPresent` do prazo comparar com a data certa mesmo que o servidor esteja em outro fuso.
- **OpenApiConfig**: título, versão e o botão **Authorize** (Bearer JWT) do Swagger.
- **RastreioApplication**: classe com `main`, que inicia tudo.

6. Banco de dados

MySQL 8.0, criado pelo **Flyway** a partir dos scripts em `resources/db/migration/`. Flyway executa cada script **uma única vez** e registra isso em uma tabela interna; ao reiniciar o Docker, o que já foi aplicado não se repete (e o volume `mysql_data` preserva os dados).

6.1 Diagrama entidade-relacionamento



Também: movimentacoes.usuario_id e perdas.usuario_id apontam para usuarios (quem registrou o evento).

6.2 Restrições que protegem os dados (V1__estrutura.sql)

Tabela	Restrições (CHECK, UNIQUE, FK)
bancas	`documento` UNIQUE.
usuarios	`email` UNIQUE; `perfil` só pode ser um dos 4 valores; **operador de banca precisa de** `banca_id` (CHECK); FK para bancas.
servicos	`valor_par > 0`.
lotes	`codigo` UNIQUE; `total_pares > 0`; `total_perdas` entre 0 e `total_pares`; `valor_par > 0`; `status` só entre os 5 valores; FKs para bancas e serviços.
lote_grade	Chave composta (`lote_id`, `numeracao`): não repete numeração; `quantidade >= 0`.
movimentacoes	FKs para lote e usuário.
perdas	`quantidade > 0`; `tipo` só entre os 3 valores.

Ou seja: **mesmo que o código Java tivesse um bug**, o banco ainda recusaria perdas maiores que o total, status inválido ou operador de banca sem banca. É uma segunda linha de defesa.

6.3 Índices

`idx_lotes_filtros (banca_id, status, criado_em)`, `idx_lotes_criado`, `idx_lotes_conclusao (concluido_em, banca_id)`, `idx_lotes_prazo (prazo, status)`, `idx_movimentos_lote` e `idx_perdas_lote`. Índices aceleram as listas, o painel e o financeiro.

6.4 Triggers (V2__proteger_historico.sql)

V2__proteger_historico.sql

```
CREATE TRIGGER movimentacoes_sem_update BEFORE UPDATE ON movimentacoes
  FOR EACH ROW SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Historico imutavel';
-- idem para DELETE em movimentacoes e UPDATE/DELETE em perdas
```

Um **trigger** é um gatilho do banco. Estes disparam **antes** de qualquer UPDATE ou DELETE nas tabelas `movimentacoes` e `perdas` e abortam a operação com erro. Assim, até um SQL digitado por engano no banco não altera o histórico. (Por isso o Compose usa `--log-bin-trust-function-creators=1`: o MySQL exige essa flag para criar triggers com esse usuário.)

> **ATENÇÃO Limite honesto da proteção**
> Quem tem poder de administrador do banco ainda consegue apagar o trigger ou a tabela. A proteção cobre a aplicação e erros operacionais, não um administrador mal-intencionado. O `VALIDACAO.md` registra isso.

6.5 Datas e fuso horário

Todas as datas-hora são gravadas em **UTC** (`DATETIME(6)`, microssegundos). A exibição converte para **Brasília** (`America/Sao_Paulo`). Nas consultas do painel, o SQL usa `TIMESTAMPADD(HOUR,-3,...)` (UTC−3) para agrupar por dia/mês local. O prazo é uma `DATE` simples (sem hora).

7. Frontend explicado, arquivo por arquivo

Caminhos relativos a `frontend/`. As telas são **componentes Vue** (`.vue`) com três partes: `<script setup>` (lógica), `<template>` (HTML) e, quando usado, estilo (aqui, tudo está em `style.css`).

7.1 Arquivos de base

Arquivo	Explicação
`index.html`	Tem só `<div id="app">` e carrega `src/main.js`. O Vue desenha tudo ali dentro (aplicação de página única, SPA).
`src/main.js`	Cria o app Vue, instala Pinia e o router e configura **dois interceptores do Axios**: um **adiciona o token** `Authorization: Bearer` em toda chamada; outro, se a API responder **401** com usuário logado, faz logout e manda para `/login`.
`src/App.vue`	Se não há token, mostra só a tela de login. Se há, mostra o layout (menu lateral, topo com nome/perfil e botão Sair). O **menu muda por perfil**: Visão geral e Valores só para gestão; Leitura só para operadores; Administração só para Admin.
`src/router.js`	Usa **hash** (`#/lotes`) para funcionar sem configuração no servidor. Cada rota pode ter `meta.roles`. O `beforeEach` redireciona para `/login` sem token e para `/lotes` se o perfil não for permitido. Esta é só uma proteção **visual**: a segurança real está na API.
`src/stores/auth.js`	Store Pinia: `token`, `usuario`, getters `perfil` e `gestao`, ações `entrar` e `sair`. O token fica **só na memória**: atualizar a página (F5) exige novo login (decisão consciente de segurança).
`src/api.js`	`api` = Axios com `baseURL: "/api"` e timeout de 15 s. `parametros(obj)` remove filtros vazios. `catalogo(nome)` busca **todas as páginas** de bancas/serviços (de 100 em 100) para preencher selects sem cortar a lista.
`src/composables/useTask.js`	Função reutilizável por todas as telas: `run(fn)` ignora clique duplo (`busy`), limpa mensagens, executa `fn`, captura erro (mostrando a `mensagem` da API e os `campos` inválidos) e libera o botão no `finally`.
`src/utils.js`	Dicionários de nomes (status, perfis, tipos de perda), `moeda` (R$ pt-BR), `numero`, `dataBR` (converte para Brasília; datas sem hora não deslocam o dia), `hoje()` e `mensagem(e)`.
`src/scanGate.js`	A **trava de leitura** (explicada no capítulo 8).
`vite.config.js`	Em desenvolvimento, repassa `/api` para `http://localhost:8080`; configura o ambiente `jsdom` dos testes.
`nginx.conf`	`try_files ... /index.html` (rotas internas); `location /api/` repassa ao backend; cabeçalhos de segurança (`nosniff`, `Referrer-Policy`, `X-Frame-Options: DENY`).

7.2 Componentes reutilizáveis (components/)

Componente	Função
`Filtros.vue`	Formulário de filtros (busca, banca, status, período). Usa `defineModel` para dois-sentidos. Carrega as bancas via `catalogo`. Operador de banca não vê o filtro de banca. No financeiro, o rótulo vira "Conclusão de".
`Grafico.vue`	Envolve o Chart.js: desenha rosca ou barras, traduz rótulos de status/tipo e se redesenha quando os dados mudam. Destrói o gráfico ao sair (evita vazamento de memória).
`Pagination.vue`	Botões Anterior/Próxima com limites e o texto "N registros · Página X de Y".
`Feedback.vue`	Caixas de erro (vermelha), sucesso (verde) e "Carregando...", com `role` de acessibilidade.

Componente	Função
`StatusBadge.vue`	Etiqueta colorida com o nome amigável do status.

7.3 Telas (views/)

Tela	O que faz e pontos de código
`LoginView.vue`	Formulário e-mail/senha. `entrar()` chama `auth.entrar` dentro do `useTask.run`; depois redireciona: **gestão →** `/`, operadores → `/leitor`.
`LotesView.vue`	Lista paginada (20 por página). `filtros` (o que está digitado) e `aplicados` (o que foi confirmado com o botão Filtrar). Botão **+ Criar lote** só para GESTOR.
`LoteFormView.vue`	Criar/editar. `grade` é uma lista de linhas {numero, quantidade}; `total` é um `computed` que soma. `salvar()` valida numeração repetida e total zero, converte a lista em mapa com `Object.fromEntries` e faz POST (novo) ou PUT (edição). Ao editar, o **serviço fica desabilitado**.
`LoteDetailView.vue`	Carrega o lote e o QR (como **blob** com `URL.createObjectURL`, liberado ao sair). Mostra dados, grade, indicadores (total, perdas, aprovados, valor), formulário de **perda** (só operador de banca em EM_PROCESSAMENTO, com `confirm`), **linha do tempo**, histórico de perdas e botões do gestor (imprimir etiqueta, editar, cancelar com motivo).
`LeitorView.vue`	Câmera + código manual (capítulo 8).
`DashboardView.vue`	Chama `/dashboard` com os filtros e **repete a cada 15 s** (`setInterval`; configurável por `VITE_POLL_MS`; o timer é limpo em `onBeforeUnmount`). Mostra 4 indicadores, faixa de status e 5 gráficos, além da lista de atrasados.
`FinanceiroView.vue`	Chama `/relatorios/financeiro` com paginação; mostra o **total devido** no filtro e a tabela banca × mês.
`CadastroView.vue`	Uma única tela serve para **bancas, serviços e usuários**: o parâmetro da rota (`:tipo`) decide a lista e o formulário. Só aceita os três tipos permitidos.

7.4 Estilo de impressão

Em `style.css`, um bloco `@media print` esconde tudo e mostra só a `.print-label` (QR, código, banca, pares), com o QR em **40 mm** (o requisito pede pelo menos 3 × 3 cm). O botão **Imprimir etiqueta** apenas chama `window.print()`.

8. Fluxos passo a passo

8.1 Login

1. `LoginView.vue` envia e-mail e senha para `auth.entrar` (Pinia).
2. `POST /api/auth/login` chega ao `AuthController.login`; o `@Valid` confere o formato do e-mail.
3. O e-mail é normalizado (`trim` + minúsculas) e buscado no banco.
4. Se não existe, está desativado ou a senha não confere com o hash BCrypt → **401** com mensagem genérica.
5. Se confere, o sistema emite um **JWT** (emissor `rastreio`, assunto = id do usuário, validade de 60 min).
6. O Vue guarda `token` e `usuario` **em memória** (Pinia) e redireciona conforme o perfil.
7. Nas chamadas seguintes, o interceptor do Axios envia `Authorization: Bearer <token>`.
8. A cada operação, `Acesso.atual()` relê o usuário no banco: desativado → 401; perfil errado → 403.

8.2 Criar o lote e gerar a etiqueta

Já descrito na seção 3.3. O ponto novo é a etiqueta: ao abrir `/lotes/<código>`, a tela baixa `GET /api/lotes/<código>/qrcode` (PNG), mostra o QR no cartão "Identificação do lote" e, ao clicar em **Imprimir etiqueta**, o CSS de impressão exibe só a etiqueta com o QR de 40 mm.

8.3 Ler o QR Code (câmera)

1. Ao abrir `/leitor`, o `onMounted` chama `iniciar()`.
2. `iniciar` confere `window.isSecureContext` (a câmera só funciona em **HTTPS ou localhost**) e cria um `Html5Qrcode`.
3. `scanner.start({facingMode: "environment"}, {fps: 10, qrbox: 230x230}, callback)` abre a câmera traseira (ou a webcam) e chama o callback **sempre que detecta um QR**. Não existe botão de captura.
4. O callback chama `processar(texto)`, que padroniza o texto (`trim` + maiúsculas) e passa pela **trava** `trava.executar(...)`.
5. Dentro, `processar` valida o formato com a regex `^L-\d{4}-[A-Z0-9-]{1,43}$`. QR estranho **nem é enviado** ao servidor.
6. Envia `POST /api/lotes/leitura/<ação>` com `{codigo}`. A ação vem do seletor: `enviar` (fábrica), `receber` ou `devolver` (banca).
7. No servidor: `LoteController.ler` → `LoteService.movimentar` (capítulo 5). Sucesso → tela **verde** com o novo status e o tempo de resposta; erro → tela **vermelha** com o motivo (ex.: "Leitura repetida ou fora de ordem. Status atual: EM_PROCESSAMENTO").
8. Se a câmera falhar ou o QR estiver danificado, o operador **digita o código** no campo manual; o formulário chama o mesmo `processar(codigo, true)` e a mesma API.
9. Ao sair da tela, `onBeforeUnmount` para e libera a câmera.

A trava de leitura (scanGate.js)

Sem ela, a câmera, a 10 quadros por segundo, leria o mesmo QR dezenas de vezes enquanto ele estiver à frente da lente. A `criarTrava` resolve duas coisas:

- **Um código só é aceito de novo após 2,5 s de ausência** (`repetido = codigo === ultimo && t - visto < 2500`). Enquanto o mesmo QR continuar visível, o tempo de "visto" é renovado e ele segue bloqueado.
- `ocupado` impede duas requisições ao mesmo tempo (mesmo de códigos diferentes). No `finally`, libera.

- A digitação manual passa `manual = true` e ignora a regra de repetição (a pessoa quis mesmo enviar).

Mesmo que a trava falhasse, o servidor protegeria: a segunda leitura da mesma etapa daria 409 (a ordem não bate) e **não criaria evento**.

8.4 Registrar perdas e devolver

1. O operador da banca abre o lote (status EM_PROCESSAMENTO). Aparece o formulário "Registrar perda antes da devolução".
2. Escolhe o tipo (danificada, extraviada, defeito de fabricação), a quantidade e a observação; confirma no `confirm(...)`.
3. `POST /api/lotes/<código>/perdas` → `LoteService.perda`: bloqueia o lote, confere status e limite, grava a perda e soma ao total.
4. A tela recarrega e mostra o novo total de perdas, aprovados e a previsão.
5. Quando termina, ele vai ao leitor, escolhe **Devolução à fábrica** e lê o QR. O lote vira CONCLUIDO, `concluidoEm` é preenchido e **novas perdas são bloqueadas**.

8.5 Cálculo do financeiro

Detalhe do lote: `valorPrevisto` = aprovados × preço contratado **a qualquer momento** (rótulo "Previsão · não consolidada"); `valorDevido` só existe quando o status é CONCLUIDO. Na tela **Valores por banca**, o `PainelService.financeiro` soma apenas lotes concluídos, agrupados por banca e por **mês da conclusão**. No exemplo oficial: 292 × R$ 2,50 = **R$ 730,00**.

8.6 Painel

`DashboardView` chama `/api/dashboard` com os filtros aplicados e repete a chamada a cada 15 s. O backend faz as contas com `COUNT/SUM/AVG` no banco. O ciclo médio considera lotes concluídos (envio → conclusão); atraso = prazo vencido e lote ainda aberto (inclui aguardando envio). Os filtros do painel usam a **data de criação**; os do financeiro, a **data de conclusão**.

8.7 Cancelar lote e cadastrar usuário

- **Cancelar**: botão só para gestor e só em AGUARDANDO_ENVIO; abre o campo de motivo (obrigatório); `POST /cancelar` → `movimentar` com `cancelar`; status CANCELADO, motivo salvo e evento no histórico.
- **Cadastrar usuário** (Admin): `CadastroView` → `POST /api/usuarios` → `CadastroService.usuario`. Operador de banca exige banca; senha mínima de 10 caracteres; senha é BCrypt; e-mail duplicado → 409.

9. Segurança, explicada

Ameaça	Defesa no projeto	Onde
Senha vazada do banco	Senhas guardadas com **BCrypt** (hash irreversível com sal). A resposta nunca devolve o hash (`@JsonIgnore`).	`Seguranca`, `Usuario`
Acesso sem login	Toda rota exige token, exceto login, health e Swagger (documentação).	`Seguranca`
Token falso ou adulterado	Assinatura HS256 com segredo de 32+ bytes; valida emissor e expiração.	`Seguranca`
Token roubado após desativar usuário	A cada operação, o usuário é relido do banco e precisa estar ativo.	`Acesso.atual`
Perfil indevido (ex.: operador criando lote)	Cada método do service chama `acesso.exigir(...)`. Esconder botão no Vue **não basta**; a API também checa.	services
Banca A vendo lote da banca B	Filtro automático por banca e verificação em detalhe, QR e mutações; responde 404 sem dados.	`Acesso.lote`, `LoteService.filtro`
Cliente mandando status ou preço	DTOs não têm esses campos; o servidor decide.	`Entradas`
SQL Injection	JPA e parâmetros nomeados (`:inicio`); nada do usuário é concatenado no SQL.	repositories, `PainelService`
Vazamento de detalhes em erro	`Erros` devolve mensagens genéricas; detalhes só no log.	`Erros`
Adulterar histórico	Sem rotas de edição/exclusão, `@Immutable`, triggers no MySQL.	`Movimentacao`, `V2`
Leitura duplicada	Máquina de estados + trava no front.	`Calculos`, `scanGate`
Segredos no código	Tudo no `.env` (fora do Git); `JWT_SECRET` com mínimo de 32 bytes validado ao iniciar.	`.env`, `application.yml`
Container comprometido	Backend roda com usuário comum (`useradd app`), não root; porta 8080 só em 127.0.0.1.	`Dockerfile`, Compose

9.1 Por que CSRF está desligado?

O CSRF é um ataque que se aproveita de **cookies de sessão** enviados automaticamente pelo navegador. Aqui não há cookie de autenticação: o token vai num cabeçalho `Authorization` que o próprio código Vue adiciona, e fica só na memória. Sem cookie, o ataque não se aplica. Também não há CORS aberto, porque o Nginx mantém tudo na **mesma origem**.

9.2 Por que o token fica só em memória?

`localStorage` pode ser lido por qualquer script injetado (XSS). Em memória, um F5 exige novo login, o que é um incômodo aceito em troca de segurança. É uma decisão documentada em `stores/auth.js`. Limite: o logout apaga o token local, mas **não invalida** um token já copiado; ele só deixa de valer ao expirar ou se o usuário for desativado.

10. Transações e concorrência

10.1 O que é uma transação

Conjunto de operações no banco tratadas como **uma só**: ou todas acontecem (*commit*), ou nenhuma (*rollback*). Imagine transferir dinheiro: debitar uma conta e creditar outra tem de ser inseparável. No sistema, "mudar o status do lote" e "gravar o evento no histórico" são o par inseparável. Isso é a anotação `@Transactional` nos métodos de `LoteService`.

10.2 Prova no teste

`AtomicidadeTest` usa o Mockito para forçar uma falha no `saveAndFlush` do histórico durante um `enviar`. O resultado esperado é HTTP 500 e o lote **continuar** AGUARDANDO_ENVIO, ou seja, a mudança de status foi desfeita.

10.3 O problema da concorrência

Duas pessoas registram perdas ao mesmo tempo num lote de 300 pares: uma de 200 e outra de 200. Se cada uma lesse "perdas atuais = 0" ao mesmo tempo, as duas passariam e o total seria 400 (impossível). Soluções usadas:

- **Bloqueio pessimista** (`PESSIMISTIC_WRITE` = `SELECT ... FOR UPDATE`): a primeira transação trava a linha do lote; a segunda **espera** até a primeira terminar e só então lê o total já atualizado (200), rejeitando a sua com 409.
- **Controle otimista** (`@Version`): cada alteração incrementa a versão do lote. Se dois salvamentos partirem da mesma versão, o segundo falha (vira 409 no `Erros`).

`ConcorrenciaTest` dispara duas threads simultâneas e verifica: **uma** perda aceita, **outra** recusada com 409, total final = 200.

10.4 Imutabilidade do histórico em três camadas

1. **API**: não há nenhuma rota para editar ou apagar eventos/perdas.
2. **JPA**: entidades `@Immutable` e colunas `updatable = false`.
3. **Banco**: triggers que abortam UPDATE e DELETE.

11. Docker

11.1 docker-compose.yml

Serviço	O que faz	Pontos importantes
mysql	Banco MySQL 8.0.	`command: --log-bin-trust-function-creators=1` (necessário para os triggers). Volume `mysql_data` guarda os dados. `healthcheck` roda um `SELECT 1` até o banco responder. `TZ: UTC`.
backend	Constrói a imagem a partir de `./backend`.	`depends_on: mysql: condition: service_healthy` (só sobe quando o banco está pronto). Variáveis: `DB_URL`, `DB_PASSWORD`, `JWT_SECRET`, `DEMO_ENABLED`, `DEMO_PASSWORD`, `TOKEN_MINUTES`. Porta `127.0.0.1:8080` (só acessível da própria máquina).
frontend	Constrói a imagem a partir de `./frontend`.	Porta `${FRONTEND_PORT:-8081}` → 80 do Nginx. Depende do backend.

O símbolo `:?` em variáveis como `${DB_PASSWORD:?Configure DB_PASSWORD no .env}` **força um erro claro** se a variável não existir (foi a mensagem que você poderia ver se esquecesse de criar o `.env`). `restart: unless-stopped` reinicia os containers sozinhos quando o Docker volta.

11.2 Dockerfiles (multi-stage)

- **backend**: etapa 1 usa a imagem do Maven para `mvn package` e gera o `.jar`; etapa 2 usa só o **JRE 17** (menor) e copia o `.jar`. Cria o usuário `app` (uid 10001) e roda com ele.
- **frontend**: etapa 1 usa Node 22 para `npm ci` e `npm run build` (gera a pasta `dist`); etapa 2 usa **Nginx** e copia só o `dist` e o `nginx.conf`.

Vantagem do *multi-stage*: a imagem final não carrega Maven nem Node, fica menor e mais segura, e o build é reproduzível em qualquer computador com Docker.

11.3 Comandos úteis

Comando (na pasta projeto-calcados)	Para que serve
`docker compose up --build`	Constrói e sobe tudo (primeira vez ou após mudar código).
`docker compose up -d`	Sobe em segundo plano, sem reconstruir.
`docker compose up --build -d backend`	Reconstrói e reinicia só o backend.
`docker compose ps`	Mostra o estado dos containers.
`docker compose logs backend --tail 100`	Últimas linhas do log do backend.
`docker compose down`	Para e remove os containers (**mantém os dados**).
`docker compose down -v`	**Apaga também o banco.** Só use para recomeçar do zero.

12. Testes

12.1 Backend (pasta src/test)

Arquivo	O que testa
`CalculosTest`	Exemplo oficial (292, 2,67%, R$ 730,00), perda total (valor zero), recusa de excesso e negativos, **matriz completa de transições** (todas as combinações status × ação) e validação de CPF/CNPJ.
`FluxoBase` (12 cenários)	Base que sobe a aplicação inteira (com H2) e chama a API por MockMvc: fluxo completo com financeiro, histórico e painel; login inválido e usuário desativado; token ausente/adulterado; isolamento entre bancas; perfis tentando burlar rotas; edição/cancelamento; repetição e ordem; **preço congelado**; perda acima do total; validação de grade, filtros e paginação; QR só com o código e reimpressão idêntica; cadastros e hash seguro.
`IntegracaoTest`	Reaproveita os cenários e acrescenta: **token expirado é recusado**.
`AtomicidadeTest`	Reaproveita os cenários e prova o **rollback** quando o histórico falha.
`ConcorrenciaTest`	Reaproveita os cenários e prova que **duas perdas simultâneas** respeitam o limite.
`MySqlTest`	Roda os cenários em um **MySQL real** (Testcontainers) e testa que UPDATE/DELETE no histórico são recusados pelo banco. Só executa com `RUN_MYSQL_TESTS=true` e Docker.

Por isso o relatório fala em "45 execuções": os 12 cenários de `FluxoBase` rodam em três contextos (integração, atomicidade, concorrência), mais os testes do `CalculosTest`. **Não são 45 cenários distintos.**

12.2 Frontend

- `utils.test.js`: formatação de data sem deslocar o dia, conversão UTC → Brasília, moeda em reais, `parametros` e a **trava de leitura** (bloqueia repetição, libera após ausência e após erro).
- `scanner.test.js`: leitor lê automaticamente e mostra sucesso, código manual usa a mesma rota e mostra erro, QR estranho não é enviado.
- `components.test.js`: mensagens, paginação, tradução de status e login guardando sessão.
- `e2e/fluxo.spec.js` (Playwright): fluxo completo com os quatro acessos em navegador real. Está escrito, mas o relatório do projeto informa que não havia sido executado.

12.3 Como rodar

Na pasta projeto-calcados (PowerShell)

```
docker run --rm -v "${PWD}/backend:/app" -w /app maven:3.9.9-eclipse-temurin-17 mvn test
docker run --rm -v "${PWD}/frontend:/app" -w /app node:22 sh -c "npm ci && npm test"
```

ATENÇÃO O que os testes provam e o que não provam

Os testes automáticos do backend usam o banco H2 (de memória). Eles **não substituem** o teste no MySQL real, na câmera física, na impressão ou nos tempos de resposta. O documento `docs/VALIDACAO.md` lista isso de forma explícita. Vocês já fizeram o teste manual do fluxo completo no Docker com MySQL real, o que cobre parte dessa lacuna.

13. O erro da primeira execução real (e a correção)

Esta é uma boa história para contar na apresentação, porque mostra **depuração real**. Se o professor perguntar se houve problemas, vocês têm uma resposta técnica sólida.

13.1 O sintoma

Ao clicar em **Novo lote**, a tela mostrava "Não foi possível concluir. Tente novamente." e o formulário não aparecia.

13.2 A investigação

1. Essa mensagem é a resposta genérica do `Erros.inesperado` (HTTP 500). Portanto, a causa real estava no **log do backend**.
2. No log (`docker compose logs backend`), a linha `Falha interna` trazia: `NullPointerException: ... because "this.s.bancas" is null`.
3. Traduzindo: o `CadastroController` tentou usar `s.bancas` (o repositório de bancas guardado dentro do `CadastroService`) e encontrou `null`.

13.3 A causa

O `CadastroService` tem métodos `@Transactional`. Para aplicar transações, o Spring **não entrega o objeto original** ao controller: entrega um **proxy** (um objeto "capa" que herda da classe). Chamadas de **método** passam pelo proxy e chegam ao objeto verdadeiro. Mas ler um **campo público** diretamente (`s.bancas`) lê o campo do proxy, que nunca foi preenchido, ou seja, `null`.

A tela "Novo lote" carrega as listas de bancas e serviços ao abrir. Como essas listagens falhavam, o formulário nunca ficava pronto.

13.4 A correção

CadastroController.java

```
// ANTES (quebrado): o controller lia um campo do service (que é um proxy)
s.bancas.findAll(f, pagina)        // s.bancas == null  ->  NullPointerException

// DEPOIS: os repositórios são injetados direto no controller
public CadastroController(CadastroService s, Acesso a,
    BancaRepository bancas, ServicoRepository servicos, UsuarioRepository usuarios)
bancas.findAll(f, pagina)          // funciona
```

Os repositórios são `@Repository` do Spring Data e podem ser injetados em qualquer classe. A lógica de negócio dos cadastros continua no service (nos métodos `banca`, `servico` e `usuario`).

13.5 Por que os testes não pegaram?

Os testes chamavam `POST /api/bancas` e `POST /api/servicos` (que usam **métodos** do service), mas nunca faziam um `GET` com sucesso em `/api/bancas` ou `/api/servicos`. Ou seja, o caminho com defeito não estava coberto. Lição: **teste de integração precisa exercitar as rotas de leitura também**, e o teste em ambiente real (Docker + MySQL) é insubstituível.

> **PERGUNTA PROVÁVEL "O que vocês aprenderam depurando?"**
> Que a mensagem de erro amigável do front esconde a causa de propósito (segurança) e que o diagnóstico está no log do servidor. Que beans transacionais são proxies e que não se deve acessar campos públicos deles de fora. E que os testes automáticos precisam cobrir também as consultas (GET), não só as gravações.

ATENÇÃO Se usarem Git

A correção está só nos seus arquivos locais. Se vocês têm um repositório Git, façam o commit do `CadastroController.java` corrigido antes de entregar, para o código entregue ser o mesmo que está rodando.

14. Perguntas e respostas para a apresentação

Respostas curtas e honestas. Ensaiem cada uma com suas palavras. As perguntas estão agrupadas por tema.

14.1 Sobre o sistema e o negócio

PERGUNTA PROVÁVEL **Qual problema o sistema resolve?**
Fábricas que terceirizam etapas do calçado perdem a visibilidade de onde está cada lote, quanto se perdeu e quanto se deve a cada banca. O sistema dá um QR Code a cada lote, registra cada etapa com responsável e horário, e calcula aprovados, perdas e valor devido automaticamente.

PERGUNTA PROVÁVEL **Quais são os perfis e o que cada um pode fazer?**
Administrador (cadastros e consultas), Gestor (cria, edita e cancela lotes, vê painel e financeiro), Operador da fábrica (registra a saída) e Operador da banca (registra recebimento, perdas e devolução; só vê a própria banca).

PERGUNTA PROVÁVEL **Como é o ciclo de vida de um lote?**
Aguardando envio → Enviado à banca → Em processamento → Concluído. Pode ser Cancelado apenas enquanto aguarda envio. A ordem é fixa e está codificada em `Calculos.transicao`.

PERGUNTA PROVÁVEL **Explique o exemplo 300 / 8 / 292.**
Lote de 300 pares com 8 perdas: aprovados = 300 − 8 = 292; perda = 8/300 = 2,67%; valor = 292 × R$ 2,50 = R$ 730,00. Está no `CalculosTest.exemploOficial`.

PERGUNTA PROVÁVEL **Por que guardar o preço dentro do lote?**
Porque o preço do serviço pode mudar. Se o lote apenas apontasse para o serviço, uma alteração de R$ 2,50 para R$ 3,00 mudaria retroativamente o valor de lotes antigos. Copiando o valor na criação, cada lote mantém o preço contratado. Há teste para isso (`precoCongeladoENaoConcluidosSemDivida`).

PERGUNTA PROVÁVEL **Qual a diferença entre valor previsto e valor devido?**
Previsto é a conta aprovados × preço feita a qualquer momento, apenas informativa. Devido só existe quando o lote está concluído: é o valor consolidado que entra no financeiro.

PERGUNTA PROVÁVEL **Por que as perdas são registradas antes da devolução?**
Porque a devolução conclui o lote e bloqueia novas perdas; o valor devido é calculado a partir das perdas finais. Quem registra é o operador da banca, que é quem tem o lote em mãos.

14.2 Arquitetura e código

PERGUNTA PROVÁVEL **Explique a arquitetura do backend.**
Camadas: o Controller recebe a requisição HTTP e valida o DTO; o Service aplica as regras, as permissões e as transações; o Repository lê e grava no banco via JPA; a Entity representa a tabela. O Vue consome a API REST por Axios.

PERGUNTA PROVÁVEL **O que é um DTO e por que usar?**
É o formato dos dados de entrada. O `LoteInput` não tem status, preço nem total, então o cliente não consegue enviar esses campos; o servidor decide. Também concentra as validações (`@NotBlank`, `@Min`, `@FutureOrPresent`).

PERGUNTA PROVÁVEL **Para que serve o `@Transactional`?**
Faz o método rodar numa transação: ou tudo é gravado ou nada. No `movimentar`, a mudança de status e o evento de histórico são gravados juntos. Se o histórico falhar, o status volta (provado no `AtomicidadeTest`).

PERGUNTA PROVÁVEL **Por que no `movimentar` não aparece `save` do lote?**
O lote foi carregado dentro da transação e é uma entidade gerenciada. O JPA detecta a alteração (dirty checking) e grava ao final.

PERGUNTA PROVÁVEL **Como o QR Code é gerado e o que ele contém?**
A biblioteca ZXing gera um PNG a partir do código do lote, com correção de erro nível H. Contém **somente o código** (nada de preço, documento ou token). Antes de gerar, o sistema confere se o usuário pode ver aquele lote.

PERGUNTA PROVÁVEL **Por que o código do lote usa UUID?**
Um contador sequencial exigiria controle de concorrência entre usuários criando lotes ao mesmo tempo. O UUID é único sem coordenação, e o `UNIQUE` do banco é a defesa final.

PERGUNTA PROVÁVEL **Por que usar `BigDecimal` no dinheiro?**
Tipos `double` têm erro de arredondamento. `BigDecimal` com 2 casas e `HALF_UP` garante os centavos corretos. No banco, a coluna é `DECIMAL(12,2)`.

PERGUNTA PROVÁVEL **Como funciona a paginação?**
O backend recebe `page` e `size` (máximo 100), consulta só aquela fatia e devolve `content`, `totalPages` e `totalElements`. O componente `Pagination.vue` mostra os botões. Isso evita carregar milhares de lotes de uma vez.

14.3 Segurança

PERGUNTA PROVÁVEL **Como funciona o login?**
O servidor confere a senha com o hash BCrypt e, se estiver certa, emite um token JWT assinado (HS256) com id do usuário, emissor e validade de 60 minutos. O Vue guarda o token na memória e o envia no cabeçalho Authorization em cada chamada.

PERGUNTA PROVÁVEL **O que é BCrypt e por que não guardar a senha em texto?**
BCrypt gera um hash irreversível com sal. Se o banco vazar, as senhas originais não são reveladas. O login compara o hash da senha digitada com o armazenado.

PERGUNTA PROVÁVEL **Esconder o botão no Vue já protege?**
Não. O Vue só melhora a navegação. Quem protege é o backend: cada método do service chama `acesso.exigir(perfil)`. Mesmo alguém chamando a API direto (Swagger, Postman) é barrado com 403.

PERGUNTA PROVÁVEL **Como impedem que uma banca veja o lote de outra?**
O filtro da lista sempre inclui a banca do usuário, e o detalhe, o QR e as mutações passam por `Acesso.lote`. O lote de outra banca responde 404, sem revelar que existe.

PERGUNTA PROVÁVEL **O que acontece se desativarem um usuário logado?**
Na próxima requisição, `Acesso.atual()` relê o usuário no banco, vê que está inativo e responde 401. O perfil também não vem do token, é reconsultado.

PERGUNTA PROVÁVEL **O sistema é vulnerável a SQL Injection?**
Não nos pontos que recebem dados do usuário: o JPA usa parâmetros e o `PainelService` usa parâmetros nomeados. O único trecho concatenado no SQL é o nome da coluna de data, que vem de constantes internas.

PERGUNTA PROVÁVEL **Por que o CSRF está desabilitado?**
CSRF explora cookies de sessão. O sistema não usa cookies: o token vai em cabeçalho, em memória. Sem cookie, o ataque não se aplica.

14.4 Banco de dados

PERGUNTA PROVÁVEL **Quem cria as tabelas?**

O Flyway, executando `V1__estrutura.sql` e `V2__proteger_historico.sql` uma vez cada. O Hibernate está em `validate`: só confere que o banco bate com as entidades.

PERGUNTA PROVÁVEL **Como garantem que o histórico não é alterado?**

Em três camadas: não existe rota de edição/exclusão; as entidades são `@Immutable`; e triggers no MySQL abortam UPDATE e DELETE nessas tabelas.

PERGUNTA PROVÁVEL **O que impede duas perdas simultâneas de passarem do total?**

O bloqueio pessimista do lote (`SELECT ... FOR UPDATE`): a segunda transação espera a primeira terminar, lê o total atualizado e é recusada com 409. Também há `CHECK(total_perdas <= total_pares)` no banco.

PERGUNTA PROVÁVEL **Por que desativar em vez de excluir cadastros?**

Porque lotes, eventos e perdas apontam para bancas, serviços e usuários. Excluir quebraria o histórico. Por isso existe o campo `ativo`.

14.5 Frontend

PERGUNTA PROVÁVEL **O que é o Pinia?**

É a biblioteca de estado global do Vue. Aqui guarda o token e o usuário logado em `stores/auth.js`.

PERGUNTA PROVÁVEL **Como o leitor evita ler o mesmo QR várias vezes?**

O `scanGate.js` bloqueia o mesmo código por 2,5 s após a última vez que foi visto e impede requisições simultâneas. Além disso, o servidor recusa a repetição da mesma etapa (409).

PERGUNTA PROVÁVEL **Por que o token fica em memória e não no localStorage?**

Para reduzir o risco de roubo por script injetado. O custo é precisar entrar de novo ao atualizar a página.

PERGUNTA PROVÁVEL **Como o painel se atualiza sozinho?**

Com `setInterval` de 15 s que repete a chamada a `/api/dashboard` com os filtros aplicados. O temporizador é limpo ao sair da tela.

14.6 Docker e testes

PERGUNTA PROVÁVEL **O que o `docker compose up` faz?**

Sobe o MySQL (espera o healthcheck), constrói e sobe o backend (Maven → JAR) e o frontend (Vite → Nginx). Os dados do banco ficam no volume `mysql_data`, e as senhas vêm do `.env`.

PERGUNTA PROVÁVEL **Por que o backend usa multi-stage?**

A etapa de build usa Maven; a imagem final leva só o JRE e o JAR, ficando menor e mais segura, e roda com usuário sem privilégios.

PERGUNTA PROVÁVEL **Como vocês testaram?**

Testes automáticos (JUnit/MockMvc no backend, Vitest no frontend) e teste manual do fluxo completo no Docker com MySQL real. O relatório `VALIDACAO.md` informa o que ainda depende de teste físico (webcam, impressão, celular, tempos).

PERGUNTA PROVÁVEL **Quais são as limitações do sistema?**

Sem recuperação de senha por e-mail (o admin redefine); token em memória (F5 pede login); CPF/CNPJ só numérico; devolução é confirmada pela banca (a confirmar com o professor); desempenho (2 s na leitura e 3 s no painel) ainda não medido; câmera em celular exige HTTPS.

PERGUNTA PROVÁVEL **O que vocês fariam a seguir?**

Executar o E2E do Playwright, medir os tempos, testar câmera e impressão em dispositivos reais, publicar com HTTPS, separar o usuário de migração do de execução no banco e adicionar testes de GET nos cadastros.

15. Limitações e pendências (seja honesto)

Estas informações vêm do `docs/VALIDACAO.md` do projeto e da nossa execução. Mostrar que vocês **conhecem os limites** do trabalho passa mais credibilidade do que prometer tudo.

15.1 O que foi realmente executado

- Compilação e testes do backend e do frontend (segundo o relatório: 45 execuções do backend e 14 do frontend aprovadas).
- Execução no **Docker com MySQL real**, login com os quatro perfis e o **fluxo completo manual** (criar lote, saída, recebimento, perdas, devolução, conferência dos valores).
- Correção do bug do `CadastroController` descoberto nessa execução.

15.2 O que continua pendente (não afirmem como feito)

- E2E Playwright em navegador (arquivo escrito, não executado).
- Webcam física, câmera de celular, leitura de etiqueta danificada e **impressão real** da etiqueta (RNF08: QR de pelo menos 3 × 3 cm).
- HTTPS publicado e medição da leitura (meta ≤ 2 s) e do painel (meta ≤ 3 s).
- `MySqlTest` (triggers e migrações em teste automatizado) e inspeção visual em desktop, tablet e celular.
- Cerimônias Scrum, publicação no GitHub e aprovação do professor.

15.3 Decisões a confirmar com o professor

- **Devolução** registrada pela banca (RF13): isso não comprova, por si só, o recebimento físico na fábrica.
- **Defeito de fabricação** desconta do pagamento? (hoje, toda perda reduz os aprovados).
- Contagem em **pares** (e não peças).
- Competência financeira pela **data de conclusão** do lote.
- Versão do Java: o projeto usa **Java 17**. Se a exigência da disciplina for Java 21, basta trocar `java.version` no `pom.xml` e a imagem base nos Dockerfiles.

16. Roteiro de demonstração e cola rápida

16.1 Antes de começar

1. Ligue o Docker Desktop e espere o motor ficar ativo.
2. Na pasta `projeto-calcados`, rode `docker compose up -d` e espere cerca de 1 minuto.
3. Abra `http://localhost:8081` e confirme o login. Tenha a senha de demonstração anotada (está no `.env`).
4. Deixe o código manual à mão: se a câmera falhar, digite o código do lote.

16.2 Roteiro sugerido (cerca de 10 a 12 minutos)

Tempo	O que mostrar	O que dizer
1 min	Tela de login	Problema e solução em uma frase; quatro perfis.
2 min	Arquitetura (capítulo 3) e DER	Vue → Nginx → Spring Boot → MySQL, tudo em Docker; camadas.
1 min	Admin: bancas, serviços, usuários	Serviço Pesponto a R$ 2,50; operador ligado a uma banca.
2 min	Gestor cria lote de 300 pares	Grade soma o total; código e QR gerados; preço copiado para o lote.
2 min	Fábrica envia; banca recebe	Máquina de estados; tentar repetir a leitura → erro 409 explicado.
2 min	Banca registra 8 perdas e devolve	292 aprovados, 2,67%, R$ 730,00; linha do tempo com 4 eventos.
1 min	Painel e Valores por banca	Indicadores no banco, atualização a cada 15 s; previsto × devido.
1 min	Segurança e testes	Banca X não vê lote da banca Y; BCrypt/JWT; testes de rollback e concorrência.

16.3 Arquivos para abrir se o professor pedir o código

Assunto	Arquivo	O que apontar
Regra do ciclo e contas	`service/Calculos.java`	`transicao`, `aprovados`, `valor` (BigDecimal).
Criar lote e preço congelado	`service/LoteService.java` → `salvar`	Soma da grade, UUID, `l.valorPar = s.valorPar`.
Leitura e histórico atômico	`service/LoteService.java` → `movimentar`, `historico`	`@Transactional`, bloqueio, `saveAndFlush`.
Concorrência	`repository/LoteRepository.java`	`@Lock(PESSIMISTIC_WRITE)` em `bloquear`.
Permissões e banca	`security/Acesso.java`	`exigir`, `lote`.
JWT e BCrypt	`security/Seguranca.java`, `AuthController.java`	Chave de 32 bytes, expiração, rotas públicas.
Leitor de QR	`LeitorView.vue`, `scanGate.js`	`Html5Qrcode.start`, regex, trava de 2,5 s.
Banco	`db/migration/V1` e `V2`	CHECK, UNIQUE, triggers.
Docker	`docker-compose.yml`	Healthcheck, volume, variáveis do `.env`.

Assunto	Arquivo	O que apontar
Testes	`CalculosTest`, `FluxoBase`	`exemploOficial`, `matrizCompletaDeTransicoes`.

16.4 Sugestão de divisão entre integrantes

- **Pessoa 1**: problema, perfis e demonstração do fluxo (capítulos 1 e 8).
- **Pessoa 2**: arquitetura, camadas e banco de dados (capítulos 3, 5 e 6).
- **Pessoa 3**: segurança, concorrência e testes (capítulos 9, 10 e 12).
- **Pessoa 4**: frontend, leitor de QR e Docker (capítulos 7, 8.3 e 11). O capítulo 13 pode ficar com quem fez a depuração.

Todos devem saber o capítulo 1 e as perguntas do capítulo 14. Ajuste conforme o tamanho do grupo.

17. Glossário

Termo	Significado simples
API REST	Conjunto de rotas HTTP (`GET`, `POST`, `PUT`) que o frontend usa para pedir e enviar dados em JSON.
JSON	Formato de texto para trocar dados entre sistemas, como `{"codigo":"L-2026-..."}`.
SPA	Aplicação de página única: o navegador carrega uma vez e o Vue troca as telas sem recarregar.
Spring Boot	Framework Java que sobe um servidor web já configurado.
Injeção de dependência	O Spring cria os objetos e os entrega prontos pelo construtor (ex.: o controller recebe o service).
Bean	Objeto criado e gerenciado pelo Spring (`@Service`, `@Component`, `@RestController`...).
Proxy	Objeto "capa" que o Spring cria por cima de um bean para adicionar comportamento (como transações).
Entity / JPA / Hibernate	Classe ligada a uma tabela; JPA é a especificação e o Hibernate é quem implementa.
DTO	Objeto só para transportar dados de entrada ou saída, com validações.
Transação	Conjunto de operações que ou acontecem todas ou nenhuma.
Commit / Rollback	Confirmar / desfazer uma transação.
Dirty checking	O JPA percebe que um objeto carregado foi alterado e grava a mudança sozinho.
Lock pessimista	Trava a linha do banco para que outros esperem (`SELECT ... FOR UPDATE`).
Controle otimista (`@Version`)	Detecta que alguém alterou o registro antes de você, em vez de travar.
Flyway / migração	Scripts SQL versionados que criam o banco, executados uma vez cada.
Trigger	Gatilho no banco que dispara antes/depois de INSERT, UPDATE ou DELETE.
JWT	Token assinado que prova quem é o usuário, com data de validade.
BCrypt	Algoritmo que transforma a senha em hash irreversível.
Bearer token	Forma de enviar o JWT no cabeçalho `Authorization: Bearer <token>`.
CSRF / XSS	Ataques pela sessão do navegador / por script injetado na página.
Pinia	Armazém de estado global do Vue.
Interceptor (Axios)	Função que roda em toda requisição ou resposta (aqui: coloca o token e trata 401).
Composable	Função reutilizável do Vue que agrupa lógica (`useTask`).
Docker / Compose	Containers (ambientes isolados) e o arquivo que descreve vários deles juntos.
Multi-stage build	Dockerfile em etapas: uma compila e a final leva só o necessário.
Volume	Pasta persistente do Docker que mantém os dados do banco entre reinícios.
Healthcheck	Teste periódico que diz se um container está pronto.
H2 / Testcontainers	Banco de memória para testes / MySQL real em container para testes.
MockMvc / Mockito	Simular chamadas HTTP em teste / simular objetos e falhas.
E2E	Teste ponta a ponta, simulando um usuário em um navegador real.